Building a Full-Stack Todo Backend with Node.js, Express & PostgreSQL

Objective

Convert a React Todo application from using `localStorage` to a production-style backend using Node.js, Express.js, PostgreSQL, and a layered architecture.

Step 1: Create Backend Project

Create a new project.

```
mkdir todo-backend
cd todo-backend
```

Initialize Node.js.

```
npm init -y
```

Install runtime dependencies.

```
npm install express pg cors
```

Install development dependency.

```
npm install --save-dev nodemon
```

Update **package.json**.

```
{
  "type": "module",
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js"
  }
}
```

Step 2: Project Structure

```
src/
│
├── config/
│     database.js
│
├── controllers/
│     todo.controller.js
│
├── repositories/
```

```
│     todo.repository.js
│
├── routes/
│     todo.routes.js
│
├── services/
│     todo.service.js
│
├── app.js
└── server.js
```

Responsibility of each folder

Routes

Maps an HTTP endpoint to a controller.

```
GET /todos
GET /todos/:id
POST /todos
PUT /todos/:id
DELETE /todos/:id
```

Controllers

- Receive HTTP requests
- Read request parameters/body
- Call services
- Return HTTP responses

Controllers should remain thin and contain minimal business logic.

Services

Contain business rules.

Examples:

- Validate due dates
- Check duplicate todos
- Trim input
- Apply default values

Services coordinate application behavior but do not communicate directly with the database.

Repositories

Contain only database queries.

Examples:

```
SELECT
INSERT
UPDATE
DELETE
```

Repositories should not know anything about HTTP requests or responses.

Config

Contains shared application configuration.

Example:

```
database.js
```

Creates a PostgreSQL connection pool used throughout the application.

Step 3: Server Entry Point

server.js

Starts the Express application.

```
import app from "./app.js";

const PORT = 5000;

app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
});
```

---

app.js

Responsible for configuring middleware and routes.

Typical order:

```
Request

↓

CORS Middleware

↓

JSON Middleware
```

```
↓

Routes

↓

404 Handler

↓

Response
```

Step 4: Middleware

Middleware executes before the request reaches a route.

Example:

```
app.use((req, res, next) => {
    console.log(req.method, req.url);
      next();
});
```

Calling `next()` passes execution to the next middleware or route.

Without `next()`, the request stops and the browser waits forever.

CORS Middleware

React runs on:

```
http://localhost:3000
```

Node.js runs on:

```
http://localhost:5000
```

Since these are different origins, browsers block requests unless the backend explicitly allows them.

Install:

```
npm install cors
```

Configure:

```
import cors from "cors";

app.use(
      cors({
        origin: "http://localhost:3000"
```

```
      })
);
```
This adds the required `Access-Control-Allow-Origin` header to responses.

JSON Middleware

`app.use(express.json());`
Automatically converts JSON request bodies into JavaScript objects.

Without this middleware:

`req.body`
would be `undefined`.

Step 5: PostgreSQL Connection

Install PostgreSQL driver.

`npm install pg`
Create `database.js`.

```
import { Pool } from "pg";

const pool = new Pool({
      user: "postgres",
      password: "password",
      host: "localhost",
      port: 5432,
      database: "todo_management"
});

export function query(text, params) {
      return pool.query(text, params);
}
```
The connection pool efficiently reuses database connections.

Step 6: Database Schema

```
CREATE TYPE priority_level AS ENUM (
      'low',
      'medium',
      'high'
);
```

```
CREATE TYPE status_type AS ENUM (
      'completed',
      'in_progress',
      'pending'
);


CREATE TABLE user_todos
(
      todo_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

      title TEXT NOT NULL,

      details TEXT NOT NULL,

      priority priority_level DEFAULT 'medium',

      status status_type Not null default 'pending',

      due_date DATE,

    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,

    updated_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);
```

Step 7: Repository Layer

Repositories communicate with PostgreSQL.

Example:

```
const result = await query(
    "SELECT * FROM user_todos ORDER BY created_at DESC"
);

return result.rows;
```

Always use parameterized queries.

Correct:

```
query(
    "SELECT * FROM user_todos WHERE todo_id = $1",
      [id]
);
```

Avoid:

```
`SELECT * FROM user_todos WHERE todo_id='${id}'`
```

Parameterized queries prevent SQL Injection.

Step 8: Understanding async/await

Database operations are asynchronous.

Repository:

```
async function allTodos() {}
```

returns

```
Promise<Array>
```

Service must also use `await`.

```
const todos = await allTodos();
```

Controller also uses `await`.

```
const todos = await allTodosService();
```

Only after the Promise resolves do we have the actual data that can be sent to the client.

Step 9: Request Flow

```
React

↓

fetch()

↓

Express Route

↓

Controller

↓

Service

↓

Repository

↓

PostgreSQL
```

```
↓

Repository

↓

Service

↓

Controller

↓

React
```

This separation keeps each layer responsible for only one concern.

Step 10: Fetching Data in React

Create a reusable service.

```
export async function getAllTodos() {

      const response = await fetch(
        "http://localhost:5000/todos"
      );

      if (!response.ok) {
      throw new Error("Unable to fetch todos");
      }

      return response.json();
}
```

Use inside React.

```
useEffect(() => {

      async function loadTodos() {

      try {

            const todos = await getAllTodos();

            setTodoItems(todos);

      } catch (error) {

            console.error(error);
```

```
      }

      }

    loadTodos();

}, []);
```

Step 11: Rendering Todos

The backend becomes the single source of truth.

```
const filteredTodos = todoItems.filter(todo => {

      const matchesStatus =
        filter.status === "all" ||
        todo.status === filter.status;

      const matchesPriority =
        filter.priority === "all" ||
        todo.priority === filter.priority;

      return matchesStatus && matchesPriority;

});
```

React only renders the data returned by the backend.

Key Learnings

- Initialize a Node.js backend with Express.
- Organize code using Route → Controller → Service → Repository architecture.
- Configure middleware for CORS and JSON parsing.
- Connect to PostgreSQL using a connection pool.
- Write safe SQL using parameterized queries.
- Understand asynchronous request handling with async/await.
- Consume REST APIs from React using `fetch`.
- Treat PostgreSQL as the application's single source of truth instead of `localStorage`.

Next Steps

- Implement `POST /todos`

- Implement `PUT /todos/:id`
- Implement `DELETE /todos/:id`
- Add server-side search, filtering, sorting, and pagination
- Add request validation
- Add authentication using JWT
- Configure environment variables and CI/CD for deployment